Full-Stack Real-Time Voting Application

Using Redux — Project Task Division

Team No: 18 | Academic Year 2024–27

Project Overview

This document outlines the division of tasks between the two team members for the development of a full-stack, real-time voting application. The system is built using Redux, React, and Node.js, following a test-first development methodology with real-time bidirectional communication via Socket.io.

Team Member 1 — Backend (Node.js + Redux Server)

1. Project Setup & Tooling

- Initialise the server project directory — voting-server/, package.json
- Configure Babel and ES6 transpilation — .babelrc, babel-preset-es2015
- Set up Mocha test scripts and test runner — package.json test & test:watch scripts
- Install Immutable.js and chai-immutable — test/test_helper.js

2. Application State Tree Design

- Design the state tree structure — entries, vote pair, tally, winner nodes
- Plan state transitions for each voting phase
- Document state shapes before coding begins

3. Core Voting Logic (Pure Functions)

- Implement setEntries() with unit tests — src/core.js
- Implement next() to begin/advance the vote — moves winning entry back to entries
- Implement vote() to tally a choice — src/core.js
- Handle vote ending and winner declaration
- Write full Mocha test coverage for all functions — test/core_spec.js

4. Redux Store — Server Side

- Write reducer combining core logic with Redux — src/reducer.js
- Create action creators (SET_ENTRIES, NEXT, VOTE) — src/action_creators.js
- Configure and initialise the Redux store — src/store.js

5. Real-Time Server (Socket.io)

- Set up Express + Socket.io server — src/server.js
- Subscribe to Redux store and broadcast state to all clients

- Receive remote action events from clients and dispatch to store
- Handle client connection and disconnection events

Team Member 2 — Frontend (React + Redux Client)

1. Client Project Setup & Tooling

- Initialise the React client project — voting-client/, package.json
- Configure Webpack for bundling — webpack.config.js
- Set up react-hot-loader for live reloading — dev server configuration
- Configure Babel for JSX and ES6 on the client
- Set up Mocha unit testing support for the client — test/ directory

2. UI Components

- Build the Voting screen (pair of choices) — src/components/Voting.jsx with unit tests
- Build the Results screen (live tally + winner) — src/components/Results.jsx
- Implement vote button interactions and disabled states
- Write unit tests for all React components — test/components/

3. Routing

- Set up React Router for /vote and /results routes — src/index.jsx
- Build the App container component — src/components/App.jsx
- Ensure each route renders the correct component

4. Redux Store — Client Side

- Write client-side reducer to apply server state — src/reducer.js
- Set up Redux store with Provider wrapper
- Connect React components to the Redux store — react-redux connect()

5. Real-Time Client (Socket.io)

- Set up Socket.io client and connect to server — src/index.jsx
- Dispatch SET_STATE action when state is received from server
- Write Redux middleware to forward VOTE actions to server — src/remote_action_middleware.js
- Wire vote button clicks through dispatch to the middleware

Shared Responsibilities

The following tasks are to be completed jointly by both team members:

- Project abstract and final report writing
- End-to-end integration testing (connecting server and client)
- Agreeing on shared contracts: action type names and state object shape

- Optional bonus exercises (invalid vote prevention, duplicate vote prevention, restart vote, socket connection indicator)

Coordination Notes

The server and client applications are designed to be independent and communicate only via WebSocket messages. This means both team members can develop and unit-test their components in parallel without blocking each other.

The only shared contract that must be agreed upon before development begins is:

- The name of the Socket.io event carrying state updates (e.g. 'state')
- The name of the Socket.io event carrying actions (e.g. 'action')
- The shape of the state object (entries list, current vote pair, tally map, winner key)